

Sistem Operasi (INF-201)

Pertemuan 7: Manajemen Memori Utama & Paging 64-bit

Tim Dosen Sistem Operasi

Program Studi Informatika – Universitas Bengkulu

Semester Ganjil 2026/2027

Tanggal Kompilasi: 8 September 2026



Capaian Pembelajaran (Sub-CPMK 7)

- Mahasiswa mampu menjelaskan arsitektur hierarki memori dan translasi Alamat Logika ke Alamat Fisik.
- Mahasiswa mampu membedakan penyebab dan solusi **Fragmentasi Internal** vs **Fragmentasi Eksternal**.
- Mahasiswa memahami mekanisme akselerasi hardware **Translation Lookaside Buffer (TLB)** dan menghitung *Effective Access Time (EAT)*.
- Mahasiswa mampu menganalisis struktur **Multi-level Paging 64-bit** (PML4 → PDPT → PD → PT).

Abstraksi Address Space (OSTEP Ch. 13)

Sistem operasi memberikan ilusi kepada setiap proses bahwa ia memiliki ruang memori raksasa, kontinu, dan privat (mulai dari alamat $0x0000$ hingga $0xFFFF...$).

Struktur Address Space Proses:

- **Text:** Kode instruksi biner program.
- **Data/BSS:** Variabel global & statis.
- **Heap:** Memori dinamis (`malloc()`).
- **Stack:** Variabel lokal & return address.

Peran MMU (Hardware):

- CPU hanya menghasilkan **Virtual Address**.
- MMU (*Memory Management Unit*) secara real-time menerjemahkan Virtual Address ke **Physical Address** di modul RAM.

Fragmentasi Internal vs Fragmentasi Eksternal

Fragmentasi Internal

Memori yang dialokasikan sedikit lebih besar daripada yang diminta proses.

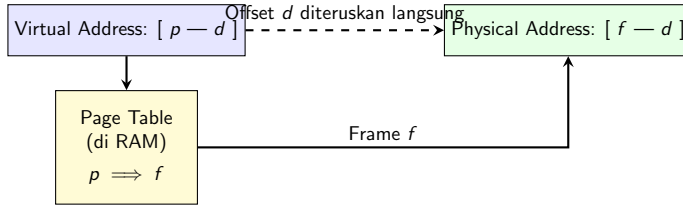
- Sisa memori di dalam blok terbuang sia-sia.
- Terjadi pada alokasi blok berukuran tetap (**Paging**).
- Contoh: Proses butuh 4097 Byte diberi 2 Page 4 KB (8192 Byte) $\Rightarrow$ **4095 Byte terbuang**.

Fragmentasi Eksternal

Total memori kosong mencukupi untuk memenuhi permintaan, namun terpecah menjadi lubang-lubang kecil tak berurutan.

- Proses baru tidak bisa masuk karena butuh ruang memori kontinu.
- Terjadi pada **Segmentasi / Dynamic Partitioning**.
- Solusi: *Compaction* (mahal) atau Paging.

Konsep Dasar Paging (OSTEP Ch. 18)



- **Page:** Pembagian ruang memori virtual menjadi blok-blok berukuran tetap (umumnya $4 \text{ KB} = 2^{12}$ Byte).
- **Frame:** Pembagian ruang memori fisik RAM dengan ukuran persis sama dengan Page.
- **Offset (d):** Menentukan posisi byte spesifik di dalam Page/Frame.

Anatomi Page Table Entry (PTE)

Setiap baris pada Page Table (*Page Table Entry*) menyimpan metadata kontrol perangkat keras:



- **Valid Bit:** 1 = page berada di RAM fisik, 0 = belum dimuat (Page Fault!).
- **Protection Bits (R/W/X):** Mengontrol apakah page boleh dibaca, ditulis, atau dieksekusi.
- **Dirty Bit:** Menandai apakah data page telah dimodifikasi (perlu ditulis ke disk swap saat diganti).

Masalah Paging: Overhead Memori 2x & Solusi TLB

Problem: Setiap Akses Memori Membutuhkan 2x Akses RAM!

1. Akses ke-1: Mengambil nomor frame f dari Page Table di RAM.
 2. Akses ke-2: Mengambil instruksi / data aktual dari alamat fisik RAM.
- ⇒ Kecepatan komputer melambat hingga 50%!

Solusi: Translation Lookaside Buffer (TLB)

- Cache perangkat keras asosiatif berkecepatan sangat tinggi (< 1 ns) di dalam chip CPU.
- Menyimpan pemetaan translasi virtual-ke-fisik yang paling sering/baru digunakan ($p \rightarrow f$).
- **TLB Hit**: Translasi selesai dalam 1 siklus CPU tanpa membaca RAM.
- **TLB Miss**: MMU membaca Page Table di RAM, lalu menyimpan hasilnya ke TLB.

Effective Access Time (EAT) Calculation

Formula Effective Access Time

$$\text{EAT} = \alpha \times (\epsilon + t_m) + (1 - \alpha) \times (\epsilon + 2 \cdot t_m)$$

di mana α = TLB Hit Ratio, ϵ = TLB Search Time, t_m = Waktu Akses Memori RAM.

Contoh Kasus Perhitungan:

- Waktu akses RAM $t_m = 100$ ns, pencarian TLB $\epsilon = 10$ ns.
- Jika $\alpha = 80\%$ (0.80):

$$\text{EAT} = 0.80 \times (10 + 100) + 0.20 \times (10 + 200) = 88 + 42 = \mathbf{130} \text{ ns}$$

- Jika $\alpha = 98\%$ (0.98):

$$\text{EAT} = 0.98 \times (10 + 100) + 0.02 \times (10 + 200) = 107.8 + 4.2 = \mathbf{112} \text{ ns (Hanya 12\% perlambatan!)}$$

Masalah Ukuran Page Table pada Arsitektur 64-bit

Mengapa Single-Level Page Table Mustahil pada Sistem 64-bit?

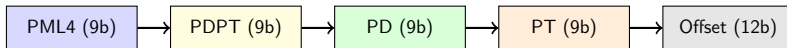
Pada sistem 64-bit dengan ukuran Page 4 KB (2^{12} Byte) dan entri PTE 8 Byte:

$$\text{Jumlah Halaman} = \frac{2^{64}}{2^{12}} = 2^{52} \text{ entri}$$

$$\text{Ukuran Page Table per Proses} = 2^{52} \times 8 \text{ Byte} = 2^{55} \text{ Byte} = \mathbf{32 \text{ Petabyte!}}$$

Fakta: Kebanyakan program hanya menggunakan sebagian kecil memori (beberapa MB saja), sehingga 99.99% entri tabel kosong tapi tetap memakan ruang RAM jika menggunakan Single-Level Table!

Solusi: Multi-Level Paging 64-bit (x86-64)



Struktur Translasi 4-Level x86-64 (48-bit Virtual Address)

- **PML4** (Page Map Level 4) → **PDPT** (Page Directory Pointer Table) → **PD** (Page Directory) → **PT** (Page Table) → **Physical RAM**.
- **Keuntungan Raksasa**: Jika suatu rentang alamat memori tidak digunakan oleh proses, seluruh sub-tree tabel di bawahnya tidak perlu dialokasikan di RAM.

Rangkuman Manajemen Memori Utama

Komponen	Fungsi Utama	Dampak Performa / Ruang
Paging 4 KB TLB Hardware	Menghilangkan fragmentasi eksternal Cache asosiatif translasi virtual-fisik	Timbul sedikit fragmentasi internal Menaikkan kecepatan akses memori mendekati 1 siklus
Multi-Level Table	Pohon hierarki translasi alamat	Menghemat ruang RAM hingga 99.9% untuk proses umum
PTE Flags	Valid bit, Read/Write, User bit	Menegakkan proteksi hak akses dan deteksi page fault

Tugas Membaca Mingguan (Persiapan UTS)

Persiapan Ujian Tengah Semester (UTS - Pertemuan 8)

Mahasiswa diwajibkan membaca ulang dan menelaah bab-bab utama dari buku teks:

- **[OSTEP]** Review Chapter 2–20 (Virtualization & Concurrency Basics)
- **[Think OS]** Review Chapter 1–9 (C Programming & Memory Systems)
- **[xv6 RISC-V]** Review Chapter 1–3 (OS Interfaces, Org, & Page Tables)

Pertanyaan Pemicu Diskusi Mandiri

Mengapa sistem operasi tidak menggunakan Page Size yang sangat besar (misal 1 GB) untuk seluruh aplikasi agar ukuran Page Table menjadi sangat kecil?